AEQDF Prototype

Adaptive Explainable Quishing Detection Framework
Complete System Architecture and Step-by-Step Build Guide

What this document is

Your paper describes *what* the AEQDF framework does. This document describes *how to actually build it* — the full architecture, the technology in every layer, the exact order of construction, and what to say when someone asks you to justify a decision in your viva. Read it once end to end before writing any code. After that, use Section 6 as your working checklist.

The system in one paragraph

A user scans or uploads a QR code in a React web app. The QR is decoded to a URL *inside the browser*, so the raw image never leaves the device. The URL is sent to an Express API, which authenticates the request and gathers evidence about that URL from six sources — the QR metadata, the URL text itself, WHOIS domain records, the SSL certificate, DNS records, and Google Safe Browsing. That evidence becomes a numeric feature vector, which is passed to a separate Python service running three trained classifiers (Random Forest, XGBoost, LightGBM). Their votes are blended into one probability, converted to a 0–100 risk score, and explained with SHAP so the user sees *why*. The verdict is stored in MongoDB and returned to the browser.

> **Viva Q:** Why is this called a *multi-stage* framework?
> **A:** Because a single QR code passes through five sequential, independently testable stages — capture, feature extraction, preprocessing, ensemble classification, and explainable prediction. Each stage has a defined input and output, so a failure can be isolated to one stage rather than debugging the whole pipeline at once.

1. System Architecture

The prototype is organised into four layers. A layer may only talk to the layer directly below it, and never reaches around it. This is what makes the system independently deployable: you can restart the ML service without touching the API, or redeploy the frontend without touching either.

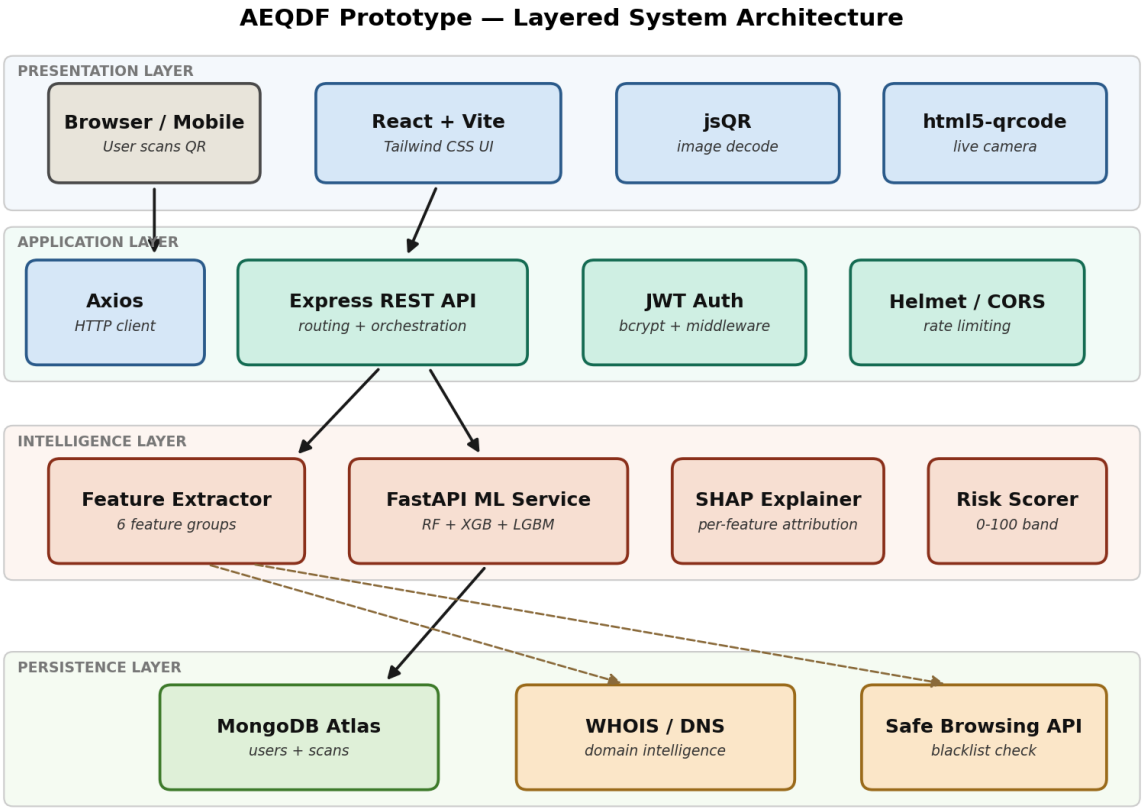


Figure A.1 — Four-layer architecture. Solid arrows are the main request path; dashed arrows are outbound calls to third-party intelligence sources.

1.1 Why four layers and not one application

The obvious alternative is a single Node application that does everything. That would be faster to write and worse in three specific ways. First, the ML libraries (scikit-learn, XGBoost, LightGBM, SHAP) are Python — running them from Node means shelling out to a subprocess on every request, which is slow and fragile. Second, model inference and API request handling have completely different resource profiles; bundling them means scaling the cheap part just to scale the expensive part. Third, a crash in the ML code would take down authentication and scan history with it.

> **Viva Q:** Why separate the ML service instead of one backend?
> **A:** Three reasons: the ML stack is Python and the API is Node, so separating them avoids cross-language subprocess calls; inference is CPU-heavy while API handling is I/O-heavy, so they scale differently; and isolating the ML service means a model crash cannot take down login or scan history. This is the standard microservice argument applied to exactly one seam, rather than splitting everything.

1.2 Layer responsibilities

Layer	Technology	Responsibility	Must NOT do
Presentation	React, Vite, Tailwind, jsQR, html5-qrcode, Axios, Recharts	Capture QR, decode to URL locally, render verdict and SHAP explanation, show history	Never call WHOIS/ML directly; never hold secrets
Application	Node.js, Express, JWT, bcrypt, Helmet, CORS, rate-limit	Authenticate, validate, orchestrate feature extraction, call ML service, persist result	Never run model inference itself
Intelligence	Python, FastAPI, scikit-learn, XGBoost, LightGBM, SHAP	Turn a feature vector into label + risk score + explanation	Never touch the database or know about users
Persistence	MongoDB Atlas, WHOIS, DNS, Google Safe Browsing	Store users and scans; supply external domain intelligence	Never contain business logic

1.3 The request lifecycle

This is the single most important diagram for your viva. If you can narrate these nine steps without notes, you can answer almost any question about how the system works.

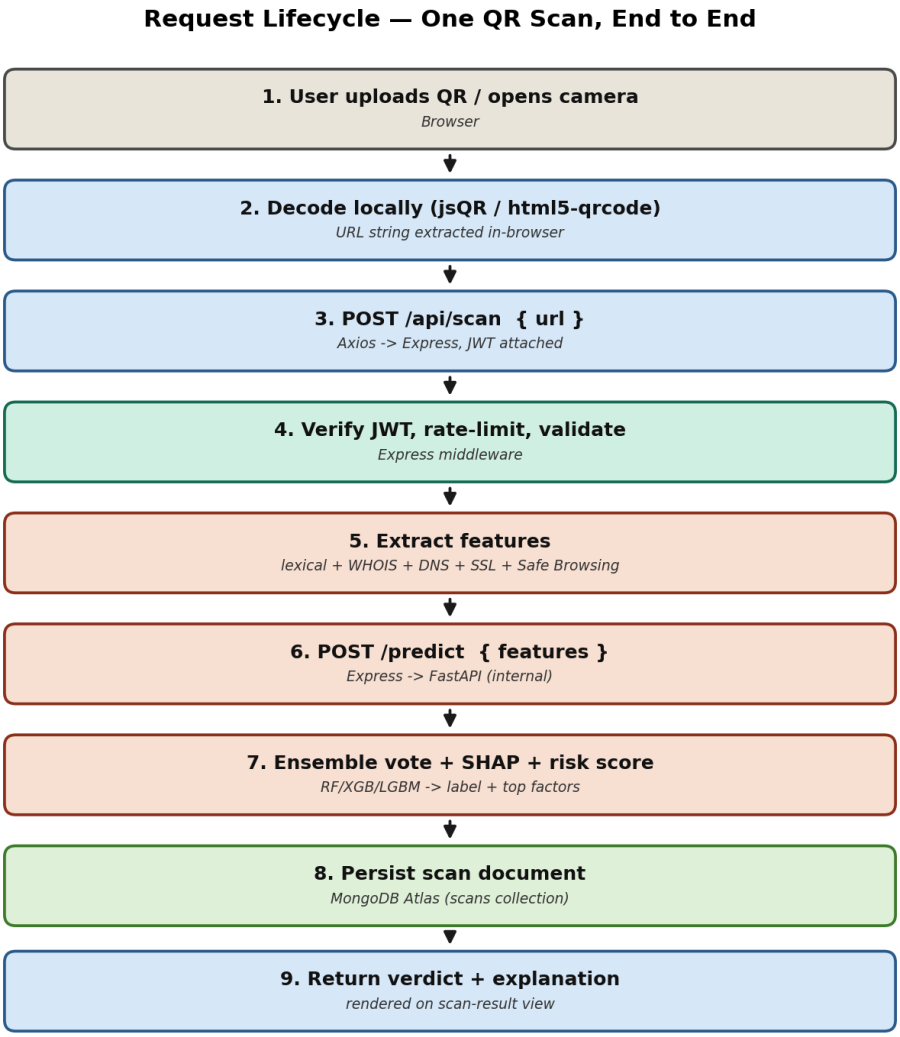


Figure A.2 — What happens on one scan, from user action to rendered verdict.

Viva Q: Why decode the QR in the browser instead of uploading the image?
A: Privacy and bandwidth. The QR image may contain more than a URL, and uploading images means storing and processing user files server-side. Decoding with jsQR in the browser means only a short URL string crosses the network, the server never handles image data, and the round trip is much smaller.

2. Data Model

Two collections. Keeping it to two is deliberate — every additional collection is another join you have to reason about, and this system has exactly two nouns: people, and the scans they performed.

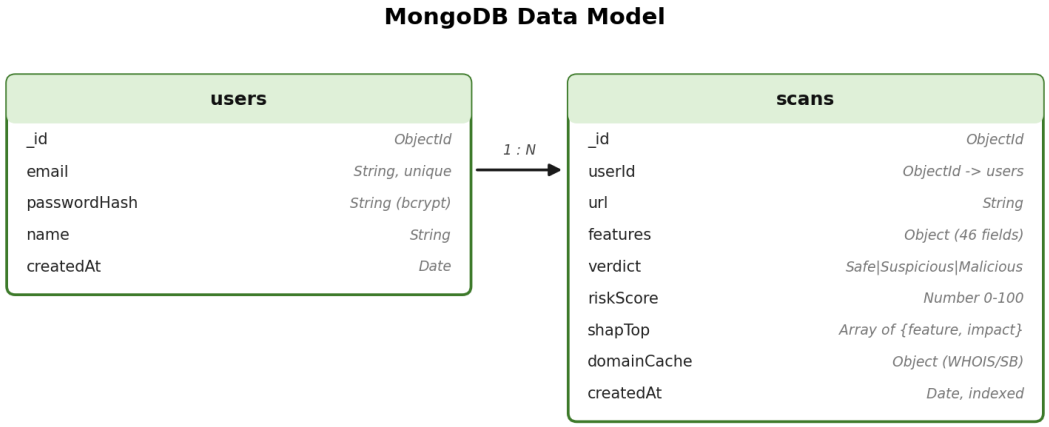


Figure A.3 — MongoDB collections and their relationship.

2.1 Design decisions worth defending

- **Passwords are never stored.** Only a bcrypt hash with a per-user salt. Even with full database access, an attacker cannot recover the original password.
- **SHAP output is stored, not recomputed.** Explanations are expensive to generate and never change for a given prediction, so they are written once at scan time.
- **Domain lookups are cached inline.** WHOIS and Safe Browsing have strict free-tier rate limits. Caching by domain means scanning fifty URLs from one domain costs one lookup.
- **createdAt is indexed.** The dashboard's most common query is 'this user's recent scans, newest first' — without the index that becomes a full collection scan as history grows.

Viva Q: Why store the SHAP explanation rather than generating it on demand?
A: SHAP values are computed against a specific model state and a specific feature vector, so they are fixed the moment the prediction is made. Recomputing them for history views would burn CPU to produce an identical answer, and would break if the model were later retrained — the stored explanation is a faithful record of why *that* decision was made at *that* time.

3. Deployment Topology

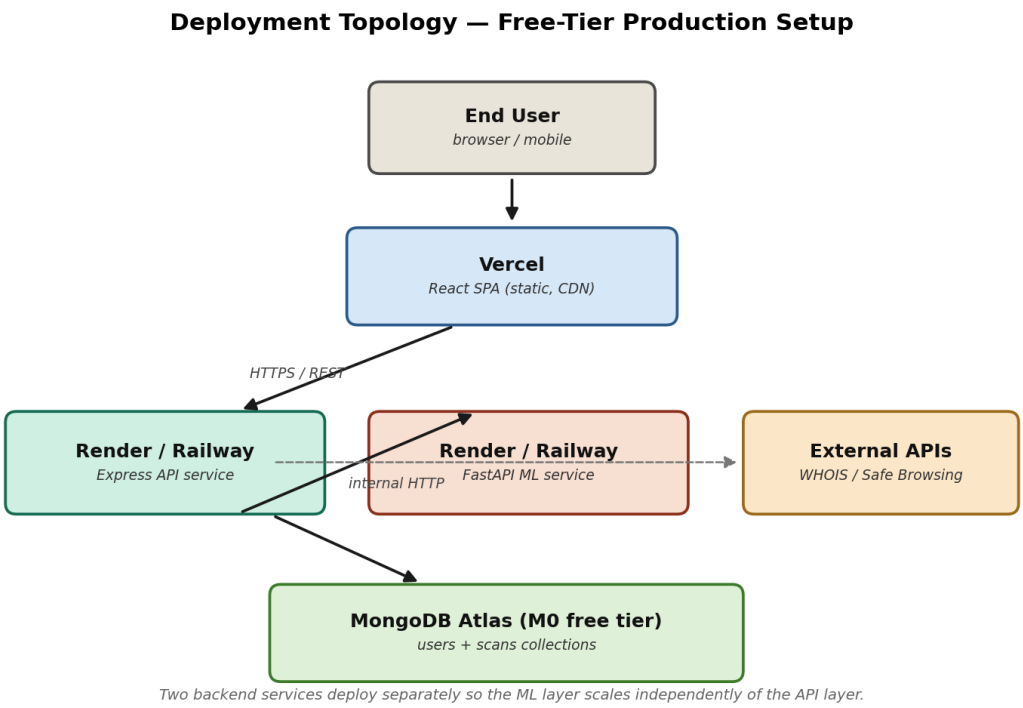


Figure A.4 — Where each piece actually runs in production.

Component	Platform	Why this choice
React frontend	Vercel	Static build served from CDN; automatic redeploy on git push; free tier is generous for SPAs
Express API	Render / Railway	Persistent Node process with environment secrets; needs to hold connections, so not serverless
FastAPI ML	Render / Railway	Separate service so model memory and restarts are isolated from the API
Database	MongoDB Atlas M0	Managed, free tier, reachable from both backend services over a secured connection string

3.1 The cold-start problem (know this before your demo)

Free tiers on Render and Railway put idle services to sleep. The first request after a period of inactivity can take 30–60 seconds while the container wakes and the model loads into memory. In a live demo this looks like your system is broken. Mitigate it by hitting a `/health` endpoint on both services a few minutes before you present, or by setting up a cron ping every ten minutes.

> **Viva Q:** What is the main weakness of this deployment?
> **A:** Free-tier cold starts and third-party rate limits. The architecture itself is sound, but WHOIS and Safe Browsing quotas cap throughput, and sleeping containers add first-request latency. Both are hosting constraints rather than design flaws — moving to paid instances and a commercial threat-intelligence feed resolves them without changing any code.

4. Build Roadmap

Build in this order. The ordering is not arbitrary — each phase produces the thing the next phase depends on, so you are never writing code against an interface you are guessing at.

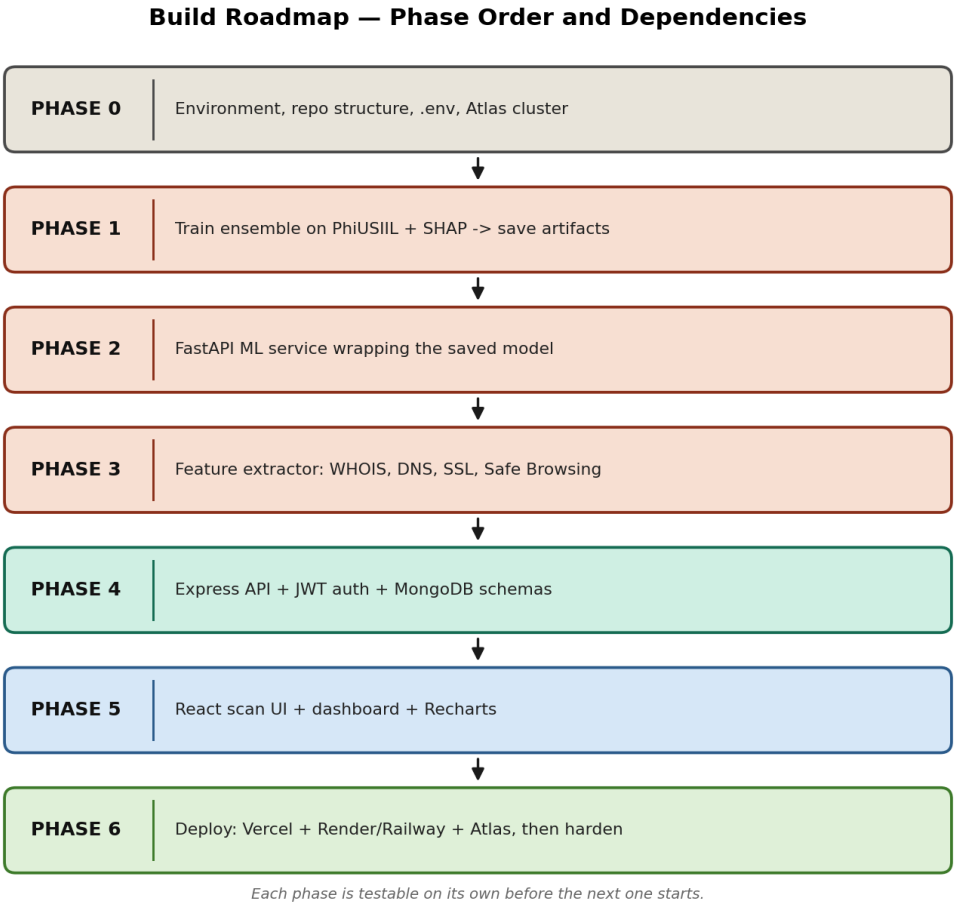


Figure A.5 — Seven phases, each independently testable.

4.1 Why the model comes first

It is tempting to build the frontend first because it is visible and satisfying. Resist this. The frontend and backend are well-understood problems that will go roughly as planned. The model is the only component that can genuinely surprise you: it may not train well, SHAP may produce explanations that do not read sensibly, or inference may be too slow to sit inside a live HTTP request. Discover that in week one, not week six.

There is also a hard dependency. Until the model exists, you do not know the exact shape of the feature vector, the label format, or the response schema — so the backend contract is guesswork and the frontend has nothing real to render. Train first and all three become settled facts.

> **Viva Q:** Your paper says the frontend was built first. Why does this guide say ML first?
> **A:** Section 4.9 of the paper describes a walking-skeleton strategy, where a deployable shell exists from day one and capabilities are layered on. That is a legitimate delivery approach and it is what was done. This guide optimises for a different goal — removing technical uncertainty as early as possible — so it front-loads the component with the highest risk. Both are defensible; the difference is deployment confidence versus technical risk.

5. Phase-by-Phase Detail

Phase 0 — Environment and repository

Goal: a clean skeleton with every service scaffolded and secrets externalised. **Done when:** all three services start locally without errors and the Atlas cluster accepts a connection.

Repository layout

```
aeqdf/
■■■ frontend/          React + Vite + Tailwind
■■■ backend/           Node + Express
■■■ ml-service/        Python + FastAPI
■   ■■■ training/      Colab/local training scripts
■   ■■■ artifacts/     saved models (git-ignored, large)
■■■ docs/              this guide, diagrams
■■■ README.md
```

Environment variables (never commit these)

```
# backend/.env
PORT=5000
MONGO_URI=mongodb+srv://<user>:<pass>@cluster.mongodb.net/aeqdf
JWT_SECRET=<long random string>
ML_SERVICE_URL=http://localhost:8000
SAFE_BROWSING_KEY=<google api key>

# ml-service/.env
MODEL_DIR=./artifacts
```

- Create the Atlas M0 cluster now — add a database user and allow your IP.
- Add `.env` and `artifacts/` to `.gitignore` on day one. Committing a secret then removing it does not remove it from git history.
- Initialise git and commit the empty skeleton before writing features.

Phase 1 — Train the ensemble (highest priority)

Goal: three trained classifiers, a weighted soft-voting ensemble, a SHAP explainer, and saved artifacts on disk. **Done when:** you can load the artifacts in a fresh Python process and get a prediction plus explanation for one sample.

Steps

- Load the PhiUSIIL dataset (235,795 rows).
- **Drop the four pre-computed similarity columns.** This is critical — they leak the label and will give you a fake 100% that collapses on real URLs.
- Stratified 80/20 split with a fixed seed (42) so results are reproducible.
- Fit a StandardScaler on the *training set only*, then apply it to the test set.
- Train Random Forest, XGBoost and LightGBM, 200 estimators each.
- Combine with weighted soft voting; tune the weights and threshold τ on validation data.
- Fit a SHAP TreeExplainer and confirm the top features are plausible.
- Save every artifact: three models, the scaler, the feature-name list, and the explainer.

Critical detail — save the feature order

The scaler and models expect features in exactly the order they were trained on. If your extraction code later produces them in a different order, you will get confident, silent, completely wrong predictions. Save the column list alongside the model and validate against it on every request.

```
joblib.dump({
    "rf": rf, "xgb": xgb, "lgbm": lgbm,
    "scaler": scaler,
    "feature_names": list(X_train.columns),   # <- order matters
    "weights": [w1, w2, w3],
    "threshold": tau,
}, "artifacts/aeqdf_ensemble.joblib")
```

Viva Q: Why did you remove four columns from the dataset?
A: They were pre-computed similarity scores derived from the label itself. Leaving them in produces label leakage — the model learns the shortcut instead of genuine structural and lexical patterns, scoring near-perfectly on the benchmark while failing on real URLs where those columns do not exist. Removing them forces the model to learn from real signals.

Phase 2 — Wrap the model in FastAPI

Goal: a running HTTP service exposing the model. **Done when:** `curl` against `/predict` returns a verdict and explanation in under a second.

Endpoints

Method	Path	Purpose
GET	/health	Liveness check; also used to warm the service before a demo
POST	/predict	Accepts a feature object, returns label, risk score, SHAP factors

Response contract — fix this now, both other layers depend on it

```
{
  "verdict": "Malicious",
  "risk_score": 92,
  "confidence": 0.987,
  "explanation": [
    {"feature": "NoOfExternalRef", "impact": -0.21,
     "reason": "Very few external references"},
    {"feature": "IsHTTPS", "impact": -0.14,
     "reason": "No HTTPS"}
  ]
}
```

- Load the model **once at startup**, not per request. Loading per request adds seconds.
- Map the 0–1 probability to 0–100 and band it: Safe 0–30, Suspicious 31–70, Malicious 71–100.
- Translate SHAP values into plain sentences. 'Domain registered 3 days ago' is useful to a user; '-0.21' is not.
- Return only the top 5–7 contributing features, not all 46.

Phase 3 — Feature extraction and domain intelligence

Goal: turn a raw URL into the exact feature vector the model expects. **Done when:** a URL string in produces a validated feature object out, with external lookups cached.

- **Lexical features** are computed directly from the URL string: length, Shannon entropy, digit ratio, special-character count, subdomain count, IP-instead-of-domain, HTTPS present, suspicious keywords. These are instant and always available.
- **WHOIS** gives domain age and registration period — a domain registered days ago is a strong phishing signal.
- **DNS and SSL** confirm the host resolves and whether the certificate is valid.
- **Safe Browsing** flags URLs already known to be malicious.

Handle failure explicitly

External lookups fail — rate limits, timeouts, privacy-protected WHOIS records. Decide the fallback for each feature *before* it happens: use a sentinel value the model saw during training, never silently substitute zero, and always set a timeout (3–5 seconds) so one slow WHOIS server cannot hang the entire request.

> **Viva Q:** What happens if the WHOIS lookup fails mid-scan?
> **A:** The extractor applies a documented fallback value for the affected features and the prediction still proceeds using the remaining signals, with the degradation recorded. This is a direct benefit of the

multi-source design — no single unavailable source can block a verdict, which would not be true for a WHOIS-only detector.

Phase 4 — Express API, authentication and persistence

Goal: a secured API that ties extraction, inference and storage together. **Done when:** you can register, log in, submit a scan and retrieve history entirely from Postman.

Endpoints

Method	Path	Auth	Purpose
POST	/api/auth/register	public	Create account, bcrypt-hash the password
POST	/api/auth/login	public	Verify credentials, issue JWT
POST	/api/scan	JWT	Extract features, call ML, persist, return verdict
GET	/api/scans	JWT	Paginated scan history for the logged-in user
GET	/api/stats	JWT	Aggregates powering the dashboard charts

Security checklist

- bcrypt with cost factor 10+; never log the plain password, not even in development.
- JWT with a genuine expiry; verify on every protected route via middleware.
- Always scope queries by `userId` from the token — never from the request body, or one user can read another's history.
- Helmet for security headers, CORS locked to your deployed frontend origin only.
- Rate-limit `/api/scan` — it triggers paid external lookups.
- Validate and normalise the incoming URL before extraction.

Phase 5 — React frontend

Goal: the interface a user actually touches. **Done when:** a person who has never seen the project can scan a QR and understand the verdict without explanation.

Pages

Page	Contents
Home	What Quishing is, what the tool does, call to action
Login / Signup	Auth forms, JWT stored, redirect on success
Scan	Upload (jsQR) or live camera (html5-qrcode); decode locally; submit URL
Result	Verdict banner, 0–100 risk gauge, plain-language SHAP factors, the URL
Dashboard	Past scans table plus Recharts summary of verdict distribution over time

The explanation is the product

This is where your paper's central claim becomes visible. Do not render raw SHAP numbers. Show ranked, readable reasons — 'Domain registered 3 days ago', 'No HTTPS', 'Unusually long URL' — with a visual indication of weight. A reviewer or examiner judging whether your explainability claim holds will judge it on this screen.

Viva Q: How does your system differ from a normal URL blacklist checker?
A: A blacklist answers only whether a URL has already been reported, so it cannot catch a domain registered this morning. AEQDF combines six independent evidence sources and a trained ensemble, so it can flag a previously unseen URL on structural grounds — and unlike both blacklists and black-box classifiers, it states which factors drove the decision.

Phase 6 — Deploy and harden

Goal: a public URL you can hand to anyone. **Done when:** the full flow works from a phone on mobile data, with no localhost anywhere.

Order of deployment

- Deploy the ML service first and confirm `/health` responds publicly.
- Deploy the Express API with `ML_SERVICE_URL` pointing at the live ML service.
- Deploy the frontend with its API base URL pointing at the live API.
- Lock CORS to the real frontend domain and remove any localhost origins.
- Restrict the Atlas IP allowlist to your backend services where possible.
- Confirm every secret is a platform environment variable, not committed in code.

Final verification

- Camera scanning requires HTTPS — verify on a real phone, not just desktop Chrome.
- Test a known-safe URL and a known-suspicious one; confirm the bands differ sensibly.
- Kill the ML service deliberately and confirm the API fails gracefully rather than hanging.
- Measure end-to-end response time on a warm service and record it — your paper claims low response time, so have the number ready.

6. Working Checklist

Print this page. Tick items as you complete them.

Phase 0 — Environment

- [ ] Repository created with frontend / backend / ml-service folders
- [ ] MongoDB Atlas M0 cluster created, user added, IP allowed
- [ ] .env files created and added to .gitignore
- [ ] All three services start locally without errors

Phase 1 — Model

- [ ] PhiUSIIL dataset loaded (235,795 rows)
- [ ] Four similarity-score columns dropped (label leakage)
- [ ] Stratified 80/20 split, seed = 42
- [ ] Scaler fitted on training set only
- [ ] RF, XGBoost, LightGBM trained (200 estimators each)
- [ ] Weighted soft voting implemented; weights and τ tuned
- [ ] SHAP explainer fitted and outputs sanity-checked
- [ ] Artifacts saved including feature-name order
- [ ] Metrics recorded: accuracy, precision, recall, F1, AUC

Phase 2 — ML service

- [ ] FastAPI app with /health and /predict
- [ ] Model loaded once at startup
- [ ] Probability mapped to 0–100 and banded
- [ ] SHAP values translated to plain-language reasons
- [ ] Response schema frozen and documented

Phase 3 — Feature extraction

- [ ] Lexical features implemented and unit-tested
- [ ] WHOIS lookup with timeout and fallback
- [ ] DNS and SSL checks implemented
- [ ] Safe Browsing integrated
- [ ] Domain-level caching working
- [ ] Feature order validated against the saved list

Phase 4 — Backend

- [ ] User and Scan schemas defined; createdAt indexed
- [ ] Register and login with bcrypt + JWT

- [ ] Auth middleware protecting scan and history routes
- [ ] POST /api/scan orchestrating extraction → ML → persist
- [ ] GET /api/scans scoped to the token's userId
- [ ] Helmet, CORS and rate limiting enabled
- [ ] Full flow verified in Postman

Phase 5 — Frontend

- [ ] Vite + React + Tailwind scaffolded, routing in place
- [ ] Login and signup wired to the API
- [ ] Image upload decoding via jsQR
- [ ] Live camera scanning via html5-qrcode
- [ ] Result page with verdict, risk gauge and readable SHAP factors
- [ ] Dashboard with history table and Recharts summary
- [ ] Loading and error states handled everywhere

Phase 6 — Deployment

- [ ] ML service deployed, /health public
- [ ] Express API deployed and pointing at live ML service
- [ ] Frontend deployed and pointing at live API
- [ ] CORS locked to production origin
- [ ] All secrets in platform environment variables
- [ ] Camera verified on a real phone over HTTPS
- [ ] End-to-end response time measured and recorded
- [ ] Demo warm-up routine documented (cold starts)

7. Viva Preparation

7.1 The two-minute narration

If you are asked to explain your project and given no further prompt, say this. It covers problem, gap, solution and evidence in the order an examiner expects.

"QR codes hide their destination until scanned, so users cannot verify a link before trusting it — this is Quishing. Existing detectors have two weaknesses: most examine only one signal, such as URL text, and almost none explain their decisions. Our framework, AEQDF, addresses both. It combines six evidence sources — QR metadata, URL lexical features, domain intelligence, SSL validation, DNS records and website behaviour — into a single feature vector, classifies it with an ensemble of Random Forest, XGBoost and LightGBM using weighted soft voting, and attaches a SHAP-based explanation plus a 0–100 risk score to every prediction. On the PhiUSIIL dataset of 235,795 URLs it reaches 99.998% accuracy with an AUC of 1.0000. The prototype is a React frontend, an Express API and a Python FastAPI inference service backed by MongoDB Atlas, deployed as independently scalable components."

7.2 Questions you should expect

Viva Q: Your accuracy is 99.998%. Is that not suspiciously high?
A: Yes, and we say so explicitly in the paper. The PhiUSIIL dataset has very high separability — cloned phishing pages systematically lack the content richness of legitimate sites, so the classes are almost linearly separable on content-structure features. The honest reading is that the number validates the pipeline, not real-world performance. Against zero-day or adversarially obfuscated attacks, accuracy would be lower. That limitation is stated in Section 4.10 rather than hidden.

Viva Q: Why three classifiers? Would one not be simpler?
A: Simpler, but less robust. The three make different kinds of errors — Random Forest bags independent trees, while XGBoost and LightGBM boost sequentially with different splitting strategies. Weighted soft voting averages their probabilities, so a confident majority can outvote one model's mistake. The cost is inference time and SHAP overhead, which is the trade-off we acknowledge.

Viva Q: What exactly does SHAP compute?
A: A SHAP value is a feature's marginal contribution to a prediction, averaged over all possible orderings in which features could be added. It comes from cooperative game theory — the Shapley value fairly distributes a payoff among contributors. Practically, it tells us how much each feature pushed this specific prediction toward malicious or legitimate, which is what turns a bare label into a reason.

Viva Q: Why is the risk score 0–100 rather than just a label?
A: A binary label forces the same response to a borderline case and a blatant one. A graded score supports triage: Safe 0–30, Suspicious 31–70, Malicious 71–100. A security team can auto-block the top band and queue the middle band for review, which a binary output cannot express.

Viva Q: What is the single biggest limitation of your work?
A: Concept drift. The model is trained on a fixed dataset snapshot, and attackers continuously change obfuscation and redirection techniques. Without a retraining mechanism, performance degrades over time. Adaptive retraining is the first item in our future work, and it is the one gap that most directly limits real deployment.

Viva Q: How would you extend this for real-world deployment?
A: Four things, in order of impact: adaptive retraining to counter drift; commercial threat-intelligence feeds instead of free-tier lookups; a mobile SDK so scanning happens in the camera app where users actually encounter QR codes; and adversarial testing with deliberately obfuscated URLs to measure robustness rather than assuming it.

7.3 Answering honestly when you do not know

If asked something you have not prepared, do not invent an answer — examiners detect it immediately and it costs more credibility than the gap itself. Say what you do know, name the boundary, and state how you would find out. 'We did not test that case; based on the architecture I would expect X, and the way to verify it would be Y' is a strong answer. Confident fabrication is not.

8. Where to Start Tomorrow

Your immediate blocker is the dataset. Everything in Phase 1 depends on having PhiUSIIL loaded, and every later phase depends on Phase 1. Concretely, in order:

1. Download the PhiUSIIL Phishing URL Dataset from the UCI Machine Learning Repository.
2. Create the repository skeleton and the Atlas cluster (Phase 0) — roughly an hour.
3. Open a Colab notebook and load the dataset; confirm 235,795 rows and inspect the columns.
4. Identify and drop the four similarity-score columns before anything else.
5. Train a single Random Forest first as a baseline. Do not build the ensemble yet — confirm the data pipeline is sound with the simplest possible model.
6. Once the baseline is sensible, add XGBoost and LightGBM and combine them.
7. Fit SHAP and read the top-ten features. If they are not plausible, something upstream is wrong.
8. Save the artifacts, then move to Phase 2.

A note on sequencing: resist building the ensemble before the single-model baseline works. If something is wrong with the data — leakage, misaligned columns, a broken split — a single Random Forest reveals it immediately and cheaply. Three models plus voting plus SHAP built on a broken foundation is significantly harder to debug.

What I can help with next: the complete Phase 1 training script (feature prep, three classifiers, weighted voting, SHAP, artifact saving) ready to paste into Colab; the FastAPI service for Phase 2; or the feature-extraction module for Phase 3. The training script is the natural next step.